FoveaWAM: Role-Aware 4D World–Action Modeling for Dynamic Manipulation Supplementary Material

Anonymous CVPR submission

A. Method and Training Details

A.1. Architecture and Training

Table S1 lists the hyperparameters of FoveaWAM on OnTimeBench.

Table S1. Hyperparameters of FoveaWAM on OnTimeBench. A training window holds 8 consecutive query positions, each with its own causal history.

Parameter	Value
Model	
Input image	224×224 RGB-D, one view
Visual encoder	MAE ViT-B/16, frozen
History length	8 steps (350 ms)
Forecast horizons	8 (50 ms–400 ms)
Summary tokens	16
Layers	12
Width	1024
Heads	16
Feed-forward width	4096
Trainable parameters	271.8M
Action	
Action dimension	7
Chunk length	8
Euler steps	5
Noise samples per query	8
Training	
Optimizer	AdamW
Betas	$(0.9,\ 0.999)$
Weight decay	10^{-4}
Learning rate	2×10^{-4}
LR schedule	cosine, 5,174 warm-up updates
Batch size	512 windows
Epochs	10 (cap: 20)
Updates / selected checkpoint	25,870 / 20,696
Gradient clipping	0.1
$\lambda_F, \lambda_P, \lambda_R, \lambda_{\text{aux}}$	0.1
Training precision	FP32, frozen encoder BF16
Hardware	64 RTX PRO 6000 GPUs, [X] h

Scene tokens. Each patch token of the frozen MAE ViT-B/16 [6] adds a learned view embedding and linear embeddings of its mean valid metric depth and of the base-frame position $\mathbf{x}_i$ of its back-projected center pixel; a patch without valid depth at its center receives no position embedding. Only the patch queries and keys receive 3D rotary embeddings from $\mathbf{x}_i$, and the same points are the source points of the sparse 3D flow.

State tokens. The 8 target-state tokens sample the offsets $-350, -300, \ldots, 0$ ms. Each is a 25-dimensional descriptor $(\mathbf{p}, \log R, \Delta\mathbf{p}, \log \Delta R, \Delta t, \mathbf{v}, \boldsymbol{\omega}, \mathbf{a}, \boldsymbol{\alpha})$: slots 0–6 describe the seven intervals between consecutive offsets, with velocities and accelerations from finite differences over the elapsed times, and slot 7 holds the current pose in the base frame. Each dimension is standardized with training statistics, invalid dimensions are zeroed, and three validity flags (pose, interval, acceleration) are appended; a slot without any valid feature is masked out of attention. The robot tokens hold the end-effector position, rotation vector, and gripper opening at the same offsets, and 16 learned embeddings form the summary tokens.

Heads and losses. Each head is a linear layer on the final world-stream states, except the target reconstruction, a two-layer MLP. The pose head reads the mean summary state ($\mathcal{L}_P$, Eq. 3). The flow head gives each patch point's displacement and visibility logit at every horizon ($\mathcal{L}_F$, Eq. 2, with scale $s = 0.715$ m and visibility weight 1); valid but invisible points still supervise their displacement. Flow labels follow the rendered surface point at each patch center through the ground-truth rigid motion of its MuJoCo geometry; a future point is visible if it projects into the image onto the same geometry within a depth tolerance of $\max(1\,\text{mm},\ 0.001z)$. $\mathcal{L}_{\text{aux}}$ sums four losses: squared errors on future metric depth (valid pixels), on the RGB of surfaces that move by more than 1 mm, and on layer-normalized DINOv3 features [12], and an absolute error on the current target's RGB region reconstructed from the mean summary state.

Action stream. Each action token embeds a seven-dimensional command: a base-frame translation increment, a left-relative rotation vector, and a gripper command, relative to the measured end-effector pose at application. Command j of query t is labeled with the demonstration command issued at $\tau_t + j\Delta_a$, since d_{task} applies only at execution (Eq. (S1)), and commands are standardized over the training

split. At deployment, the gripper changes state only when its command leaves a deadband of 0.5 in Ball capture and 0.1 in every other task, including Ball capture, table.

A.2. Pretraining on DROID and Fine-Tuning

The real-robot base trains on DROID [8] (15 Hz) with the first exterior camera and its depth, using the 3D point tracks and calibrations released by PointWorld [7]. Its 42,087 usable episodes yield 598,889 anchor windows, 60,149 of them for validation; each anchor expands into 8 query positions as on OnTimeBench, and only the anchor carries flow and DINOv3 labels. Each patch follows one PointWorld track that starts inside it, preferring the one with the largest consistent motion so that small moving objects are covered; 47% of the point–horizon slots carry a label.

DROID provides no object poses, so the base has no target-state tokens or pose head; it trains the action loss, the sparse 3D flow with $\lambda_F = 1$ and no visibility term, and the future DINOv3 features with weight 0.1. It has 48 layers at width 1024 (1.02B trainable parameters) and trains for 20 epochs (21,060 updates) at a global batch of 512 on 64 GPUs with the optimizer and precision of Table S1. Each real-robot policy is fine-tuned from it on its task's demonstrations [updates, learning rate, and batch size], with the target-state tokens and missing heads freshly initialized; no real-robot policy uses simulation data or weights.

B. OnTimeBench and Evaluation Protocol

B.1. Benchmark Design

The tasks run in MuJoCo 3.11.0 and robosuite 1.5.2 [13, 18] with a Franka Panda arm and a parallel gripper under robosuite's operational-space pose controller, with a control period of 50 ms and a physics step of 2 ms. A fixed third-person camera with a 40° vertical field of view renders 224×224 RGB-D images. An action is an end-effector translation and rotation-vector increment in the base frame with a gripper command, clipped to 0.10 m per translation axis and 0.5 rad per rotation component. Table S2 lists the fourteen tasks.

B.2. Scenarios, Demonstrations, and Splits

Each scene samples the target's initial state and motion parameters uniformly within task-specific ranges: besides the speeds of Table S2, harmonic amplitudes of up to 35 mm at 0.25–0.7 Hz, yaw rates of up to 1.2 rad/s, wire rotation rates of 2.5–7.5°/s, and ramp slopes of 5–15°. In an occluded scene, the target is hidden for 0.5–3.0 s and then reappears. Only physically feasible scenes are kept, and evaluation replays the stored scenes.

The expert is a task-specific closed-loop state machine with privileged simulator state that anticipates the target's motion, for example by predicting the interception point in the capture tasks. A demonstration is kept if the expert succeeds, a replay of its actions reproduces the episode, and its path and jerk pass quality checks. The training pool holds 11,200 demonstrations, 800 per task, recorded at 20 Hz with RGB-D, robot states, commands, and simulator ground truth, and 350 further episodes form the validation split; policies receive only the inputs of Sec. 3.

We select configurations on 1,400 development scenes (100 per task) and report all OnTimeBench results on 1,400 held-out scenes shared by all methods, generated with the same ranges from seeds used in no training, validation, or development episode.

B.3. Real-Time Evaluation and Metrics

Executor. All methods share one executor on the 50 ms control clock. Command j of query t is meant for $\tau_t + j\Delta_a$, and its execution time (Sec. 3.1) is

$$\tau^{\mathrm{a}}_{t,j} = \tau_t + j\Delta_a + d_{\mathrm{task}}, \tag{S1}$$

where $d_{\mathrm{task}} = 50$ ms for the post-impact grasp, which applies each command 50 ms after it is issued, and $d_{\mathrm{task}} = 0$ otherwise. As in RTC [2], latency counts in whole control periods: a query with total latency L_t hands over its chunk at the first boundary at or after it ends, $k_t = \lceil L_t/\Delta_a \rceil$ periods after capture, and execution continues from command k_t, the one meant for that boundary. The policy then observes again and starts the next query; meanwhile the previous chunk continues, so a policy replans as often as its latency allows. A chunk holds the method's native number H_m of commands (Table S3) and is dropped if $k_t \geq H_m$. Before the first chunk arrives and whenever a chunk runs out, the controller holds the current arm target and gripper command; no command is repeated. The executor runs asynchronously, as in the streaming of DynamicVLA; no method, including FoveaWAM, inpaints across chunks as in RTC.

Protocols. In real time, physics advances during each query by the method's latency, drawn per query from its distribution measured on one RTX 5090, while the rollouts themselves run on RTX PRO 6000. With paused physics, inference takes no physics time and a policy replans every 50 ms; this removes the wait for inference but also raises the feedback rate, so it is a reference rather than an upper bound. The latency sweep adds 0 to 200 ms in steps of 50 ms after inference in real time.

Latency measurement. We time each method alone on one RTX 5090 at batch size 1 in each baseline's release precision and in compiled BF16 for FoveaWAM, over three rounds of 30 warm-up and 300 measured calls on recorded observations with the required causal history. A timed query starts from the newest observation and ends when its first command is available. It includes all computation that needs

Table S2. **The fourteen OnTimeBench tasks.** Target: the body whose pose the policy receives (Sec. 3.1); for placement, the receptacle, and for passage, the wire entry, the rest of the wire being context. Speed: target speed range (capture tasks: at the ramp exit). Grasp: bilateral fingerpad contact, a lift of at least 0.05 m, and a continuous 0.5 s hold. An episode fails at unsafe contact, leaving the workspace, a drop, or the time limit (Limit). Group (Fig. 6): R, rotation-sensitive (the target rotates during the lead and its orientation matters at contact); C, context-active (an obstacle clearance or a driver: conveyor, ramp, or striker).

Task	Code	Target	Speed (m/s)	Success	Limit (s)	Group
Harmonic grasp	`grasp`	block	0.03–0.20	grasp	8	R
Conveyor grasp	`t1_belt`	part	0.04–0.10	grasp	8	C
Translating grasp	`t1_xyz`	part	0.04–0.15	grasp	8	–
Rotating grasp	`t1_so3`	part	0.04–0.15	grasp	8	R
Occluded grasp	`t1_occ`	part	0.04–0.10	grasp without canopy contact	8	C
Moving receptacle	`t2_trans`	receptacle	0.03–0.06	held object at rest inside	10	–
Rotating receptacle	`t2_se3`	T-shaped receptacle	0.03–0.06	held T-part at rest inside	10	R
Wire passage	`t3_phase`	wire entry	[X]	align within 30 mm, pass without contact	21	R, C
Full wire passage	`t3_full`	wire entry	[X]	grasp wand, pass, exit, release	32	R, C
Ball capture	`t4_sphere`	ball	[0.4–0.7]	grasp before the tray	6	C
Ball capture on table	`t4_sphere_tabletop`	ball	[0.4–0.7]	grasp while rolling	6	C
Slider capture	`t4_slider`	slider	[0.4–0.8]	grasp while sliding	6	C
Can capture	`t4_can`	can	[0.4–0.7]	grasp before tipping	6	R, C
Post-impact grasp	`t5_replan`	part	0.12–0.14	grasp along its axis	6	R, C

the newest frame, such as preprocessing, image encoding, the online flow and motion descriptors of PUMA and DynamicWAM, sampling, and action decoding, and on the robot also tracking. The forecast heads of FoveaWAM are skipped at deployment and excluded from time-to-action. A query misses its deadline when this time-to-action exceeds 50 ms, and the action age of a command is its execution time minus the capture time. Appendix C.2 reports the measured values.

Metrics. SR weights the 14 tasks equally. The position error at first contact e_p is the distance of the end effector from the expert's contact frame on the target; in the placement tasks, which start with the object held, it uses the contact of the held object with the receptacle and the expert's placement pose. e_p averages over the 13 tasks with a contact, excluding `t3_phase`, and over the episodes that make contact; failures without contact remain in the SR denominator. Paired 95% intervals bootstrap held-out scenes within each task, jointly across methods.

B.4. Baselines

Common conditions. Each baseline keeps its official architecture, chunk length, initialization, and training recipe (Table S3), trains on the same 11,200 demonstrations mapped to its action space through the same Cartesian controller, and is configured on development scenes. Rollouts use the agent view each baseline was trained with, any wrist slot masked, and the histories of Table 1; masking the wrist view changes p95 latency by at most 3.8 ms in the seven configurations timed both ways. DP uses its authors' real-robot DDIM sampler with 16 steps, and OpenWAM-α runs with video decoding disabled and is timed with and without its official cache and compilation.

Table S3. **Baseline training.** Updates × batch: training budget. H: commands per returned chunk. Params: trainable parameters, frozen encoders excluded. DOM ckpt.: the released DynamicVLA checkpoint trained on DOM.

Method	Initialization	Updates×batch	H	Params
ACT [17]	scratch	50,000 × 32	100	[X]
DP [4]	scratch	50,000 × 32	15	[X]
$\pi_{0.5}$ [1]	`pi05_base`	30,000 × 256	10	[X]
$\pi_{0.5}$+SE(3)	`pi05_base`	30,000 × 256	10	[X]
DynamicVLA [16]	DOM ckpt.	50,000 × 44	20	[X]
PUMA [5]	official	[X]×[X]	[X]	[X]
OpenWAM-α [14]	official	34,490 × 192	32	[X]
DynamicWAM [10]	official	40,000 × 128	16	[X]
DynamicWAM+SE(3)	official	40,000 × 128	16	[X]
FoveaWAM	scratch	25,870 × 512	8	271.8M

SE(3)-augmented baselines. $\pi_{0.5}$+SE(3) and DynamicWAM+SE(3) receive the eight target-state tokens of Appendix A.1 from the same pose source, with the same validity flags. A two-layer MLP with a zero-initialized output layer maps them to the model's width; for $\pi_{0.5}$ they follow the image tokens in the prefix, where the action expert attends to them, and for DynamicWAM they join its kinematic descriptors. Training follows the base model's recipe.

C. Additional Simulation Results

C.1. Per-Task Results

Table S4 uses the Native real-time protocol with +0 ms extra delay. Every method fails on rotating-receptacle placement. FoveaWAM reaches 38% on moving receptacle placement

Table S4. **Per-task held-out success** (%) on OnTimeBench for every method of Table 1, Native real-time protocol: each method's measured inference latency plus **+0 ms extra delay**, with physics continuing during inference; 100 paired scenes per task. FoveaWAM uses compiled BF16 C (p95 15.1 ms). DP: Diffusion Policy. DVLA: DynamicVLA. OWAM: OpenWAM-α with official cache/compile; OW-off: both disabled. DWAM: DynamicWAM. +: with our SE(3) history tokens. Δ: FoveaWAM minus the strongest other method on that task; ties follow table order. The task-balanced row compares against the strongest overall baseline, PUMA. Brackets give pointwise 95% paired scene-bootstrap intervals, conditional on the selected comparator and checkpoint (100,000 resamples; no selection adjustment). The last two rows give first-contact position error e_p (mm), averaged over the 13 contact tasks, and the number n of episodes with a contact (Appendix B.3); –: no complete 13-task mean. Δ shading is blue for higher SR and orange for lower, saturating at 15 points. Blue/bold and light blue mark the best and second-best displayed values.

Task	ACT	DP	$\pi_{0.5}$	$\pi_{0.5}$+	DVLA	PUMA	OWAM	OW-off	DWAM	DWAM+	**Ours**	Δ [95% CI]
Harmonic grasp	49.0	31.0	51.0	85.0	39.0	85.0	41.0	1.0	50.0	51.0	**95.0**	+10.0 [3.0, 17.0]
Conveyor grasp	45.0	45.0	96.0	58.0	89.0	89.0	62.0	0.0	59.0	47.0	**100.0**	+4.0 [1.0, 8.0]
Translating grasp	14.0	33.0	59.0	38.0	61.0	51.0	53.0	0.0	53.0	15.0	**89.0**	+28.0 [18.0, 38.0]
Rotating grasp	24.0	20.0	32.0	44.0	26.0	67.0	42.0	3.0	30.0	37.0	**92.0**	+25.0 [14.0, 36.0]
Occluded grasp	30.0	28.0	37.0	51.0	80.0	84.0	34.0	0.0	44.0	36.0	**97.0**	+13.0 [6.0, 21.0]
Moving receptacle	26.0	8.0	26.0	19.0	**39.0**	0.0	28.0	0.0	0.0	0.0	38.0	-1.0 [-16.0, 14.0]
Rotating receptacle	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**	+0.0 [0.0, 0.0]
Wire passage	99.0	99.0	**100.0**	**100.0**	99.0	87.0	50.0	58.0	23.0	23.0	**100.0**	+0.0 [0.0, 0.0]
Wire passage, full	0.0	3.0	1.0	5.0	**12.0**	0.0	0.0	0.0	0.0	0.0	10.0	-2.0 [-10.0, 6.0]
Ball capture	36.0	16.0	46.0	21.0	41.0	39.0	45.0	13.0	24.0	23.0	**76.0**	+30.0 [20.0, 41.0]
Ball capture, table	71.0	81.0	**100.0**	**100.0**	88.0	**100.0**	99.0	88.0	98.0	**100.0**	**100.0**	+0.0 [0.0, 0.0]
Slider capture	56.0	30.0	58.0	68.0	28.0	76.0	71.0	31.0	47.0	63.0	**80.0**	+4.0 [-6.0, 14.0]
Can capture	91.0	31.0	98.0	99.0	72.0	**100.0**	**100.0**	5.0	96.0	**100.0**	**100.0**	+0.0 [0.0, 0.0]
Post-impact grasp	42.0	5.0	49.0	51.0	39.0	57.0	21.0	22.0	43.0	20.0	**87.0**	+30.0 [20.0, 41.0]
Task-balanced	41.6	30.7	53.8	52.8	50.9	59.6	46.1	15.8	40.5	36.8	**76.0**	+16.4 [14.3, 18.5]
e_p (mm)	32.2	36.2	31.2	32.5	30.0	27.2	31.7	–	31.2	37.0	**25.2**	–
n	1110	941	1231	1187	1203	1272	1115	439	1193	970	1295	–

versus DynamicVLA's 39%, and 10% on full wire passage versus DynamicVLA's 12%; both paired intervals include zero. It ties the strongest baseline on phase-only wire passage, tabletop ball capture and can capture. The largest recorded failure classes for FoveaWAM are rim impulse violations (160 of 1,400 scenes) and wire contacts (55 scenes).

C.2. Latency

Table S5 lists all eleven configurations at five added latencies; Fig. 5 plots all 55 accepted points.

Table S5. **Success versus added latency** on 1,400 held-out OnTimeBench scenes per point (SR in %, real-time protocol). The 0 and 150 ms columns equal the Native and +150 ms columns of Table 1. Blue/bold and light blue mark the best and second-best values per column.

	Added latency (ms)				
Method	0	50	100	150	200
ACT	41.6	43.5	36.5	24.3	16.5
Diffusion Policy	30.7	26.1	21.0	15.9	14.3
$\pi_{0.5}$	53.8	47.2	39.6	27.8	13.6
$\pi_{0.5}$ + SE(3)	52.8	39.8	32.4	28.2	15.1
DynamicVLA	50.9	46.4	30.6	23.4	17.8
PUMA	59.6	53.2	36.4	**31.1**	**27.1**
OpenWAM-α	46.1	36.1	27.9	24.7	22.1
OpenWAM-α (off)	15.8	13.7	14.2	10.4	9.5
DynamicWAM	40.5	27.6	25.0	20.6	20.9
DynamicWAM + SE(3)	36.8	30.6	23.9	21.9	21.3
FoveaWAM	**76.0**	**63.4**	**42.8**	25.6	9.8

Timing. Table S6 breaks one query of FoveaWAM down by stage.

Table S6. **Latency breakdown** of one query of compiled BF16 FoveaWAM on one RTX 5090 at batch size 1 (ms, p50 / p95), and action age on the robot. The policy total uses 900 measured queries and excludes pose tracking; magenta dashes mark pending measurements.

Stage	p50	p95
Pose tracking (per frame, robot)	—	—
Observation encoding + H2D	4.83	5.28
World stream (once per query)	3.93	4.22
Action stream (5 Euler steps)	4.27	4.41
Policy time-to-action (total)	14.66	15.65
Training-only readouts (for reference)	0.37	0.41
Action age at execution (robot)	—	—
Queries over 50 ms (robot, %)	—	

C.3. Results on DOMINO

We use the 24 single-arm Franka tasks of the clean level-1 setting of DOMINO [5] with its demonstrations, scenes, and native evaluator. The manipulation score (MS) scales route completion by penalties for leaving the workspace or view and for collisions. FoveaWAM reads the simulator pose of the target named by the instruction and masks slots without a valid pose (Appendix A.1); the RGB-based baselines do not receive this pose.

Data and adaptation. All methods share 1,200 training and 240 validation demonstrations, execute absolute joint-position targets with DOMINO's native controller, and adapt their state and action projections to its eight-dimensional joint space; normalizers use training data only. The official sampler of DynamicWAM skips demonstrations too short for its windows, which leaves it 1,054 training and 210 validation demonstrations and none for two tasks; all 24 tasks remain in the test set. DynamicWAM trains for 40,000 updates at global batch 128. $\pi_{0.5}$ starts from its official base, trains for 30,000 updates at global batch 256, and is evaluated with its final EMA. PUMA keeps its published weights and frozen teacher and trains at global batch 64 with early stopping on validation action MSE, which selected update 6,000. [FoveaWAM: initialization, updates, batch size, and checkpoint selection.]

C.4. Ablation Controls

Each of the six controls of Sec. 4.3 (Fig. 6) changes one component of the configuration of Table S1 and keeps the data, the recipe with its checkpoint selection, and the executor. Removing a loss also removes its head, and the control without target history removes target-state slots 0–6 and keeps slot 7, the current pose (Appendix A.1). The uniform fine flow control removes $\mathcal{L}_P$ and the pose head and supervises the flow at [4×4] points in each patch, a [56×56] grid, with labels from the same ground-truth rigid motion.

C.5. Robustness to Pose Noise

With frozen weights, we evaluate three synthetic perturbations of the target pose: nominal temporally correlated noise, twice its amplitude, and nominal noise with contiguous missing observations. The nominal process combines a fixed per-episode bias and a stationary correlated component, each with per-axis standard deviations of 5 mm and 1°, with a correlation time of 200 ms. Dropouts last 200 ms, separated by geometrically distributed valid gaps with a mean of 3.8 s, giving a long-run missing fraction of 5%. The perturbation applies to the current and past estimates before the history features are computed.

On the same held-out scenes with native latency (Fig. S1), SR changes by [X.X], [X.X], and [X.X] points relative to clean poses under these three perturbations.

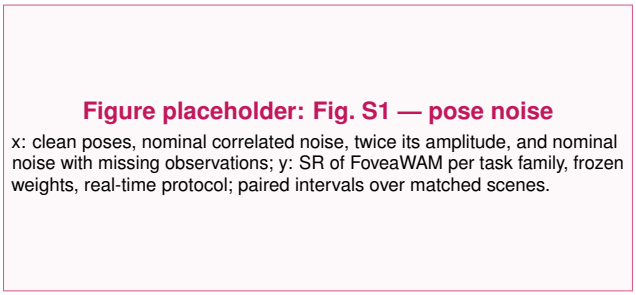


Figure S1. **Robustness to pose noise.** SR of FoveaWAM under synthetic perturbations of the target pose, relative to clean poses.

D. Real-Robot Experiments

D.1. Platform and Tasks

The robot is a Franka arm ([model]) with [gripper model] and a fixed RealSense D435 RGB-D camera, commanded through [a timestamped Cartesian interface] at [control frequency]; a fixed third-person camera records RGB-D at [resolution and frame rate], calibrated by [procedure] before each session; baselines whose recipes use a wrist view also receive [wrist camera]. The task devices are installed on this platform one at a time (Fig. 7; Table S7).

D.2. Pose Tracking and Labels

Tracking is marker-free and uses one metric mesh per object instance; a ball is defined by its center and radius. In the first frame, Grounding DINO [9] localizes the object named by the task, SAM 2.1 [11] segments it, and FoundationPose [15] registers its pose; afterwards, Cutie [3] propagates the mask and FoundationPose refines a Kalman-filter prediction at each update. An empty mask makes the estimate invalid rather than carrying the previous pose forward, and [re-initialization after a loss]. The tracker updates on the

Table S7. **Real-robot tasks.** Target: the tracked object. Trial key: what each of the 20 test keys fixes. Speed: target speed from the recorded RGB-D of all test trials (m/s, mean ± s.d.). Grasp: the object held between both fingers and lifted by [X] cm. Each task has 100 training and 20 validation demonstrations.

Task	Target	Motion	Trial key	Speed	Success	Limit (s)
T1	block	belt conveyor	[placement on the belt, belt speed]	—	grasp	[X]
T2	open box	belt conveyor	[box placement, belt speed]	—	held block stays in box	[X]
T3	block, unequal sides	turntable	[radius, initial angle and yaw, angular speed]	—	grasp across short sides	[X]
T4	toy car	remote driving, fixed throttle	[start pose, heading]	—	grasp	[X]
T5	tennis ball	[mechanism]	[release position, motion setting]	—	grasp	[X]
T6	block on cart	inclined rail	[slope, release point]	—	grasp	[X]

Figure placeholder: Fig. S2 — real-robot failures

(a) Failures by category (≈35% width): one horizontal stacked bar per method ($\pi_{0.5}$, DynamicWAM, ACT, FoveaWAM), length = failed test trials of 120, segments in the fixed order late, misaligned, slip/drop, collision or safety stop, timeout, tracking lost. **(b) One paired trial key** (≈65%): two rows of five unretouched third-person frames, FoveaWAM (success) above and [baseline] (late) below, at the same times after the first command; gripper closure marked.

Figure S2. **Real-robot failures.** (a) Failures by category over the 120 test trials of each method. (b) One trial key of [task]: FoveaWAM grasps the target, whereas [baseline] closes its gripper behind it (late). The key is one on which the baseline fails in its most frequent category; frames are taken at the same times after the first command.

newest frame at every control-period boundary and keeps that frame's capture time, from which the target-state tokens take their finite differences.

The target-state inputs of a demonstration replay the same tracker causally over its recorded RGB-D. The pose labels are the tracked poses at future boundaries, [refined offline over the episode], and the flow labels follow [3D point tracks of the recorded RGB-D, as for DROID (Appendix A.2)]. For the ball, the target-state tokens and the pose head use translation only. Over all test trials of FoveaWAM, the estimate was invalid at [X]% of control-period boundaries, with a longest gap of [X] s and [X] re-initializations[, most of them on task].

D.3. Training and Evaluation Protocol

The baselines keep the initialization, recipe, and chunk length of Table S3, and FoveaWAM is fine-tuned as in Appendix A.2; DynamicWAM executes [with or without real-time chunking]. Demonstrations, development trials, and test trials come from separate sessions.

Each task has 20 pre-generated trial keys that fix the object instance, its initial pose, and the device setting. For each key, the four methods run consecutively in a random order, so that drift of the apparatus does not align with the methods, after an operator resets the scene against a reference image. A trial ends at success, failure, a safety stop, or a time limit of about three times the task's median demonstration duration. A key is repeated only if the device deviates from it on its own ([speed outside ±X% of the setting or a device fault]); everything else, including a tracking loss of FoveaWAM, counts as a failure. Table S6 also gives the action age and the missed deadlines on the robot.

D.4. Results and Failure Analysis

Table S8. **Real-robot success with intervals.** All: success rate over the 120 test trials of each method (%) with its 95% Wilson interval. Δ: paired difference from FoveaWAM (points), with a bootstrap interval over trial keys within each task. Per-task rates are in Table 3.

Method	All [95% CI]	Δ [95% CI]
$\pi_{0.5}$	—	—
DynamicWAM	—	—
ACT	—	—
FoveaWAM	—	–

Table S8 gives the pooled success rates with their intervals, and Fig. S2 the failures. The failure categories, fixed be-

fore the test trials, are observable events: late (the gripper closes behind the target), misaligned grasp, slip or drop after contact, collision or safety stop, timeout, and tracking lost (FoveaWAM only). [The dominant failure of each method, and what it shows about latency; the most frequent failure of FoveaWAM.]

References

[1] Kevin Black, Noah Brown, James Darpinian, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Proceedings of The 9th Conference on Robot Learning*, pages 17–40, 2025. 3

[2] Kevin Black, Manuel Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. In *Advances in Neural Information Processing Systems*, pages 33383–33407, 2025. 2

[3] Ho Kei Cheng, Seoung Wug Oh, Brian Price, Joon-Young Lee, and Alexander Schwing. Putting the object back into video object segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024. 5

[4] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin C. M. Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. 3

[5] Heng Fang, Shangru Li, Shuhan Wang, Xuanyang Xi, Dingkang Liang, and Xiang Bai. Towards generalizable robotic manipulation in dynamic environments. In *European Conference on Computer Vision*, pages 314–333, 2026. 3, 5

[6] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022. 1

[7] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. PointWorld: Scaling 3D world models for in-the-wild robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20765–20779, 2026. 2

[8] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024. 2

[9] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In *European Conference on Computer Vision*, 2024. 5

[10] Yunfan Lou, Hewen Gao, Xiyu Zhu, Zhuoran Qiao, Xuan Han, Yifan Yang, Yifan Ye, Boxian Yao, and Zhibo Pang. DynamicWAM: Dual-path motion conditioning for world-action models in dynamic manipulation. *arXiv preprint arXiv:2608.00793*, 2026. 3

[11] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. In *International Conference on Learning Representations*, pages 28085–28128, 2025. 5

[12] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, Federico Baldassarre, Maxime Oquab, Cijo Jose, Vasil Khalidov, Marc Szafraniec, Seungeun Yi, Michaël Ramamonjisoa, et al. DINOv3. *arXiv preprint arXiv:2508.10104*, 2025. 1

[13] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. 2

[14] Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. OpenWAM: An open, modular exploration towards systematic world-action model pretraining. *arXiv preprint arXiv:2609.07398*, 2026. 3

[15] Bowen Wen, Wei Yang, Jan Kautz, and Stan Birchfield. FoundationPose: Unified 6D pose estimation and tracking of novel objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17868–17879, 2024. 5

[16] Haozhe Xie, Beichen Wen, Jiarui Zheng, Zhaoxi Chen, Fangzhou Hong, Haiwen Diao, and Ziwei Liu. DynamicVLA: A vision-language-action model for dynamic object manipulation. In *Advances in Neural Information Processing Systems*, 2026. Accepted for publication. 3

[17] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, 2023. 3

[18] Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Kevin Lin, Abhiram Maddukuri, Soroush Nasiriany, and Yifeng Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020. 2